

MC23CA901 - PROBLEM SOLVING AND PROGRAMMING IN C

LIST OF EXPERIMENTS

S. No.	Name Of The Experiment	Page No.	Marks	Signature
1	Design Algorithm, Flowchart, and Pseudocode for Real-World Problem Scenarios	4		
2	C programs covering variables, data types, and operators	7		
3 A	Decision-Making Statements	9		
3. B	Find the day of the week using switch statement	11		
3. C	Display multiplication table using for loop	13		
3. D	Calculate sum of natural numbers using while loop	15		
3. E	Demonstrate do-while loop (Menu-driven example)	17		
3. F	Demonstrate break and continue	19		
4. A	Implementation of Arrays	21		
4. B	Implementation of String Manipulation Operations	27		
5	Pointer	31		
6	User-Defined Functions with Different Parameter Passing Techniques	34		
7	Dynamic Memory Management using malloc(), calloc(), and free() in C	37		
8	Structure	41		
9	File operations: Reading, Writing, and Modifying a File using C	44		

Course Objectives

1. To introduce students to the fundamentals of computational thinking and structured problem-solving techniques.
2. To implement these solutions efficiently using the C programming language
3. To emphasize structured programming principles, and the use of arrays, functions, and pointers for writing optimized code.
4. To accentuate structured programming principles, and the use of arrays, functions, and pointers for writing optimized code.
5. To gain hands-on experience in memory management and file handling in C.

Course Outcomes

1. Understand computational thinking and problem-solving techniques.
2. Design algorithms and represent those using flowcharts and pseudocode.
3. Develop C programs using structured programming principles.
4. Apply concepts of arrays, pointers, and functions to write efficient programs.
5. Implement memory management techniques and file handling operations in C.

Ex. No. : 1	Design Algorithm, Flowchart, and Pseudocode for Real-World Problem Scenarios

AIM

To design an algorithm, draw the corresponding flowchart, write the pseudocode, and implement a C program for a given real-world problem scenario — *for example, finding whether a person is eligible to vote based on age.*

PROBLEM STATEMENT

Check whether a person is eligible to vote or not based on the entered age.

ALGORITHM

1. **Start**
2. Read the value of `age`.
3. If `age` is greater than or equal to 18,
 Display “Eligible to Vote”.
 Otherwise, display “Not Eligible to Vote”.
4. **Stop**

PSEUDOCODE

BEGIN

 INPUT `age`

 IF `age >= 18` THEN

 PRINT "Eligible to Vote"

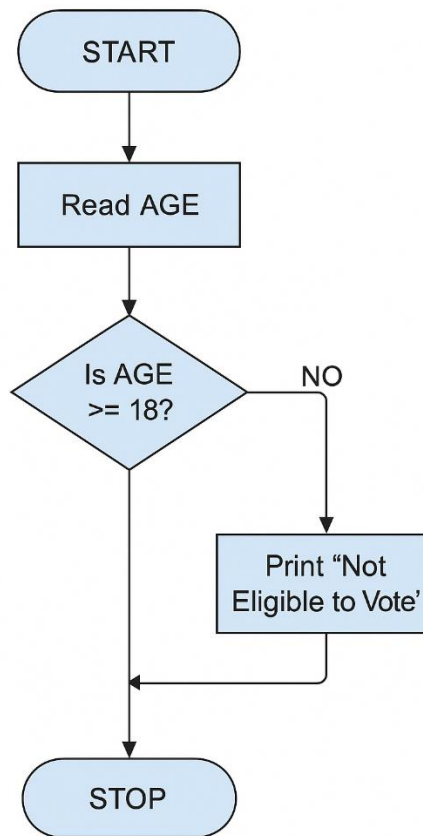
 ELSE

 PRINT "Not Eligible to Vote"

 ENDIF

END

FLOWCHART



PROGRAM

```
#include <stdio.h>

int main() {

    int age;

    printf("Enter your age: ");

    scanf("%d", &age);

    if (age >= 18)

        printf("Eligible to Vote\n");

    else

        printf("Not Eligible to Vote\n");
```

```
    return 0;

}
```

SAMPLE INPUT AND OUTPUT

Input	Output
20	Eligible to Vote
15	Not Eligible to Vote

RESULT

Thus, the algorithm, flowchart, pseudocode, and C program for checking voting eligibility based on age were successfully designed and implemented.

Ex. No. : 2	C Program Using Variables, Data Types, and Operators

AIM

To write and execute a C program that demonstrates the use of variables, data types, and arithmetic operators.

ALGORITHM

1. Start the program.
2. Declare variables of different data types (e.g., int, float, char).
3. Initialize the variables with suitable values.
4. Perform arithmetic operations such as addition, subtraction, multiplication, and division.
5. Display the results using the `printf()` function.
6. Stop the program.

PROGRAM

```
#include <stdio.h>

int main() {

    int a = 10, b = 5;      // Integer variables

    float x = 2.5, y = 1.5; // Float variables

    char ch = 'A';         // Character variable

    int sum = a + b;

    int diff = a - b;

    int prod = a * b;

    float div = (float)a / b; // Type casting for division

    printf("Character value: %c\n", ch);

    printf("Sum of %d and %d = %d\n", a, b, sum);
```

```
printf("Difference of %d and %d = %d\n", a, b, diff);

printf("Product of %d and %d = %d\n", a, b, prod);

printf("Division of %d and %d = %.2f\n", a, b, div);

printf("Float addition: %.2f + %.2f = %.2f\n", x, y, x + y);

printf("Float multiplication: %.2f * %.2f = %.2f\n", x, y, x * y);

return 0;

}
```

SAMPLE OUTPUT

Character value: A

Sum of 10 and 5 = 15

Difference of 10 and 5 = 5

Product of 10 and 5 = 50

Division of 10 and 5 = 2.00

Float addition: 2.50 + 1.50 = 4.00

Float multiplication: 2.50 * 1.50 = 3.75

RESULT

The program successfully demonstrates the use of different **data types**, **variables**, and **arithmetic operators** in C.

Ex. No. : 3. A	Decision-Making Statements

AIM

To check whether a person is eligible to vote (if-else).

ALGORITHM

1. Start the program.
2. Read the person's age.
3. If $\text{age} \geq 18$, print "Eligible to vote".
4. Else, print "Not eligible to vote".
5. Stop the program.

PROGRAM

```
#include <stdio.h>

int main() {

    int age;

    printf("Enter your age: ");

    scanf("%d", &age);

    if (age >= 18)

        printf("You are eligible to vote.\n");

    else

        printf("You are not eligible to vote.\n");

    return 0;

}
```

SAMPLE INPUT

Enter your age: 20

Sample Output

You are eligible to vote.

RESULT

The **control flow statement, if-else** was successfully implemented and executed using simple real-world examples.

Ex. No. : 3. B	Find the day of the week using switch statement

AIM

To find the day of the week using switch statement.

ALGORITHM

1. Start the program.
2. Read an integer between 1 and 7.
3. Use a `switch` statement to print the corresponding day.
4. Stop the program.

PROGRAM

```
#include <stdio.h>

int main() {

    int day;

    printf("Enter day number (1-7): ");

    scanf("%d", &day);

    switch(day) {

        case 1: printf("Monday\n"); break;

        case 2: printf("Tuesday\n"); break;

        case 3: printf("Wednesday\n"); break;

        case 4: printf("Thursday\n"); break;

        case 5: printf("Friday\n"); break;

        case 6: printf("Saturday\n"); break;
```

```
        case 7: printf("Sunday\n"); break;

        default: printf("Invalid day number!\n");

    }

    return 0;

}
```

SAMPLE INPUT

Enter day number (1-7): 5

SAMPLE OUTPUT

Friday

RESULT

The **control flow statement**, `switch` was successfully implemented and executed using simple real-world examples.

Ex. No. : 3 C	Display multiplication table using for loop

AIM

To display multiplication table using for loop.

ALGORITHM

1. Start the program.
2. Read an integer *n*.
3. Use a `for` loop to print the multiplication table of *n* from 1 to 10.
4. Stop the program.

PROGRAM

```
#include <stdio.h>

int main() {

    int n, i;

    printf("Enter a number: ");

    scanf("%d", &n);

    for(i = 1; i <= 10; i++) {

        printf("%d x %d = %d\n", n, i, n * i);

    }

    return 0;

}
```

SAMPLE INPUT

Enter a number: 5

SAMPLE OUTPUT

5 x 1 = 5

5 x 2 = 10

...

5 x 10 = 50

RESULT

The **control flow statement** `for` were successfully implemented and executed using simple real-world examples.

Ex. No. : 3 D	Calculate sum of natural numbers using while loop

AIM

To Calculate sum of natural numbers using while loop

ALGORITHM

1. Start the program.
2. Read the value of n.
3. Initialize sum = 0 and counter = 1.
4. Repeat while counter $\leq$ n: add counter to sum.
5. Print sum.
6. Stop.

PROGRAM

```
#include <stdio.h>

int main() {

    int n, i = 1, sum = 0;

    printf("Enter the limit: ");

    scanf("%d", &n);

    while(i <= n) {

        sum += i;

        i++;

    }

    printf("Sum of first %d natural numbers = %d\n", n, sum);

    return 0;
```

```
}
```

SAMPLE INPUT

Enter the limit: 5

SAMPLE OUTPUT

Sum of first 5 natural numbers = 15

RESULT

The **control flow statement** `while` was successfully implemented and executed using simple real-world examples.

Ex. No. : 3 E	Demonstrate do-while loop (Menu-driven example)

AIM

To demonstrate do-while loop (Menu-driven example)

ALGORITHM

1. Start the program.
2. Display a menu for basic arithmetic operations.
3. Perform the selected operation.
4. Ask the user if they want to continue.
5. Repeat using do-while.
6. Stop.

PROGRAM

```
#include <stdio.h>

int main() {

    int a, b, choice;

    char ch;

    do {

        printf("\n1.Addition\n2.Subtraction\n3.Multiplication\n4.Division\nEnter your choice: ");

        scanf("%d", &choice);

        printf("Enter two numbers: ");

        scanf("%d%d", &a, &b);

        switch(choice) {
```

```
    case 1: printf("Sum = %d\n", a + b); break;

    case 2: printf("Difference = %d\n", a - b); break;

    case 3: printf("Product = %d\n", a * b); break;

    case 4: printf("Quotient = %.2f\n", (float)a / b); break;

    default: printf("Invalid choice\n");

}

printf("Do you want to continue? (y/n): ");

scanf(" %c", &ch);

} while(ch == 'y' || ch == 'Y');


return 0;

}
```

RESULT

The **control flow statement** `do-while` was successfully implemented and executed using simple real-world examples.

Ex. No. : 3 F	Demonstrate break and continue

AIM

To write a program that demonstrates the use of `break` and `continue` statements using a simple loop structure.

ALGORITHM

1. **Start the program.**
2. **Initialize** a variable `i` for looping (e.g., `i = 1`).
3. **Display a message** indicating that the program will demonstrate `break` and `continue`.
4. **Use a `for` or `while` loop** that runs from 1 to 10.
 - For example: `for(i = 1; i <= 10; i++)`
5. **Inside the loop:**
 - **Check a condition for `continue`:**
 - If `i` equals 5, use the `continue` statement to skip the current iteration.
 - This means when `i = 5`, it will not execute the rest of the loop body and move to the next iteration.
 - **Check a condition for `break`:**
 - If `i` equals 8, use the `break` statement to terminate the loop completely.
6. **If neither condition is true**, display the current value of `i`.
7. **End the loop.**
8. **Display a message** indicating the end of the program.
9. **Stop.**

PROGRAM

```
#include <stdio.h>

int main() {

    int i;

    for(i = 1; i <= 10; i++) {

        if(i == 5)
```

```
        continue; // Skip 5

    if(i == 9)

        break;    // Stop loop

    printf("%d ", i);

}

return 0;

}
```

Sample Output

1 2 3 4 6 7 8

RESULT

The **control flow statements** `break`, and `continue` were successfully implemented and executed using simple real-world examples.

Ex. No. : 4 .A	Implementation of Arrays

AIM

To write a C program to implement and perform basic operations on arrays such as insertion, deletion, searching, and traversal.

ALGORITHM

Step 1:

Start the program.

Step 2:

Declare an array and necessary variables such as n, i, pos, value, etc.

Step 3:

Input the number of elements and store them in the array.

Step 4:

Display the array elements (Traversal).

Step 5:

Perform the following operations:

- **Insertion:**
 - Read the position and value.
 - Shift elements to the right.
 - Insert the new value at the specified position.
- **Deletion:**
 - Read the position to delete.
 - Shift elements to the left to fill the gap.
- **Searching:**
 - Read the element to be searched.
 - Traverse the array to check if the element exists.

Step 6:

Display the modified array after each operation.

Step 7:

Stop the program.

PROGRAM

```
#include <stdio.h>

int main() {

    int arr[100], n, i, pos, value, choice, search, found = 0;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    printf("Enter %d elements:\n", n);

    for (i = 0; i < n; i++)

        scanf("%d", &arr[i]);

    printf("\nArray elements are:\n");

    for (i = 0; i < n; i++)

        printf("%d ", arr[i]);

    printf("\n");

    printf("\n1. Insert\n2. Delete\n3. Search\nEnter your choice: ");

    scanf("%d", &choice);

    switch (choice) {

        case 1:
```

```

printf("Enter position and value to insert: ");

scanf("%d%d", &pos, &value);

if (pos < 0 || pos > n)

    printf("Invalid position!");

else {

    for (i = n; i > pos; i--)

        arr[i] = arr[i - 1];

    arr[pos] = value;

    n++;

    printf("Array after insertion:\n");

    for (i = 0; i < n; i++)

        printf("%d ", arr[i]);

    }

    break;

case 2:

    printf("Enter position to delete: ");

    scanf("%d", &pos);

    if (pos < 0 || pos >= n)

        printf("Invalid position!");

    else {

        for (i = pos; i < n - 1; i++)

            arr[i] = arr[i + 1];

```

```

        n--;

        printf("Array after deletion:\n");

        for (i = 0; i < n; i++)

            printf("%d ", arr[i]);

    }

    break;

case 3:

    printf("Enter element to search: ");

    scanf("%d", &search);

    for (i = 0; i < n; i++) {

        if (arr[i] == search) {

            printf("Element found at position %d\n", i);

            found = 1;

            break;

        }

    }

    if (!found)

        printf("Element not found!\n");

    break;

default:

    printf("Invalid choice!");

```

```
}  
  
return 0;  
  
}
```

SAMPLE INPUT AND OUTPUT

INPUT 1 (INSERTION)

Enter number of elements: 5

Enter 5 elements:

10 20 30 40 50

1. Insert

2. Delete

3. Search

Enter your choice: 1

Enter position and value to insert: 2 25

Output 1:

Array after insertion:

10 20 25 30 40 50

INPUT 2 (DELETION)

Enter number of elements: 5

Enter 5 elements:

10 20 30 40 50

1. Insert

2. Delete

3. Search

Enter your choice: 2

Enter position to delete: 3

Output 2:

Array after deletion:

10 20 30 50

Input 3 (Search):

Enter number of elements: 5

Enter 5 elements:

10 20 30 40 50

1. Insert

2. Delete

3. Search

Enter your choice: 3

Enter element to search: 40

Output 3:

Element found at position 3

RESULT

Thus, the C program to implement and perform basic operations on arrays (insertion, deletion, searching, and traversal) was successfully executed and verified.

Ex. No. : 4 B	Implementation of String Manipulation Operations

AIM

To write a C program to perform basic string manipulation operations such as finding string length, string concatenation, string comparison, string copy, and string reverse without using built-in functions.

ALGORITHM

Step 1: **Start the program.**

Step 2: **Declare two character arrays `str1` and `str2`.**

Step 3: **Input two strings from the user.**

Step 4: **Perform the following string operations manually using loops:**

- **Length:** Count characters until `'\0'`.
- **Concatenation:** Append second string to the first.
- **Comparison:** Compare each character of both strings.
- **Copy:** Copy each character from source to destination.
- **Reverse:** Print string characters in reverse order.

Step 5: **Display the result of each operation.**

Step 6: **Stop the program.**

PROGRAM

```
#include <stdio.h>
```

```
int main() {
```

```
    char str1[100], str2[100], str3[100];
```

```
    int i, j, len = 0, cmp = 0;
```

```
    printf("Enter first string: ");
```

```
    gets(str1);
```

```
    printf("Enter second string: ");
```

```

gets(str2);

// String Length

for (i = 0; str1[i] != '\0'; i++);

len = i;

printf("\nLength of first string: %d", len);

// String Concatenation

for (i = 0; str1[i] != '\0'; i++);

for (j = 0; str2[j] != '\0'; j++, i++)

    str1[i] = str2[j];

str1[i] = '\0';

printf("\nAfter concatenation: %s", str1);

// String Comparison

for (i = 0; str1[i] != '\0' && str2[i] != '\0'; i++) {

    if (str1[i] != str2[i]) {

        cmp = 1;

        break;

    }

}

if (cmp == 0)

```

```
    printf("\nStrings are Equal");

else

    printf("\nStrings are Not Equal");

// String Copy

for (i = 0; str2[i] != '\0'; i++)

    str3[i] = str2[i];

str3[i] = '\0';

printf("\nCopied String: %s", str3);

// String Reverse

printf("\nReversed String 1: ");

for (i = len - 1; i >= 0; i--)

    printf("%c", str1[i]);

printf("\n");

return 0;

}
```

SAMPLE INPUT AND OUTPUT

Enter first string: Hello

Enter second string: World

OUTPUT

Length of first string: 5

After concatenation: HelloWorld

Strings are Not Equal

Copied String: World

Reversed String 1: dlroWolleH

RESULT

Thus, the C program to perform string manipulation operations such as length, concatenation, comparison, copy, and reverse was successfully executed and verified.

Ex. No. : 5	Pointer

AIM

To write a C program to demonstrate the basic concepts of pointers such as pointer declaration, initialization, accessing variable values using pointers, and pointer arithmetic.

ALGORITHM

Step 1: Start the program.

Step 2: Declare integer variables and pointer variables.

Step 3: Initialize the variable with a value.

Step 4: Assign the address of the variable to the pointer using the address-of (&) operator.

Step 5: Display the value of the variable using both the variable name and the pointer (using dereference operator *).

Step 6: Perform pointer arithmetic (increment and decrement) and observe the address changes.

Step 7: Stop the program.

PROGRAM

```
#include <stdio.h>

int main() {

    int a = 10, b = 20;

    int *ptr1, *ptr2;

    // Pointer Initialization

    ptr1 = &a;

    ptr2 = &b;

    // Displaying values and addresses

    printf("Address of a: %p\n", &a);

    printf("Address stored in ptr1: %p\n", ptr1);
```

```
printf("Value of a using pointer: %d\n", *ptr1);

printf("\nAddress of b: %p\n", &b);

printf("Address stored in ptr2: %p\n", ptr2);

printf("Value of b using pointer: %d\n", *ptr2);

// Pointer Arithmetic

printf("\nPointer Arithmetic Demonstration:\n");

printf("Address in ptr1 before increment: %p\n", ptr1);

ptr1++;

printf("Address in ptr1 after increment: %p\n", ptr1);

ptr1--;

printf("Address in ptr1 after decrement: %p\n", ptr1);

return 0;

}
```

SAMPLE INPUT AND OUTPUT

Output:

```
Address of a: 0x7ffeece5a5ac

Address stored in ptr1: 0x7ffeece5a5ac

Value of a using pointer: 10

Address of b: 0x7ffeece5a5a8

Address stored in ptr2: 0x7ffeece5a5a8

Value of b using pointer: 20
```

Pointer Arithmetic Demonstration:

Address in ptr1 before increment: 0x7ffeece5a5ac

Address in ptr1 after increment: 0x7ffeece5a5b0

Address in ptr1 after decrement: 0x7ffeece5a5ac

RESULT

Thus, the C program demonstrating pointer declaration, initialization, dereferencing, and pointer arithmetic was successfully executed and verified.

Ex. No. : 6	User-Defined Functions with Different Parameter Passing Techniques

AIM

To write a C program to demonstrate user-defined functions using different parameter passing techniques:

1. Call by Value
2. Call by Reference

ALGORITHM

Step 1: Start the program.

Step 2: Define two functions:

- `callByValue(int a)` – to demonstrate passing arguments by value.
- `callByReference(int *b)` – to demonstrate passing arguments by reference.

Step 3: In the `callByValue()` function, modify the value of the parameter and print it.

Step 4: In the `callByReference()` function, modify the value using the pointer variable and print it.

Step 5: In the `main()` function:

- Declare integer variables.
- Call both functions with appropriate arguments.
- Display the values before and after function calls.

Step 6: Observe how the original values are affected by each technique.

Step 7: Stop the program.

PROGRAM

```
#include <stdio.h>

// Function for Call by Value

void callByValue(int x) {

    x = x + 10;
```

```

    printf("Inside callByValue function: x = %d\n", x);

}

// Function for Call by Reference

void callByReference(int *y) {

    *y = *y + 10;

    printf("Inside callByReference function: y = %d\n", *y);

}

int main() {

    int a = 5, b = 5;

    printf("Before callByValue: a = %d\n", a);

    callByValue(a);

    printf("After callByValue: a = %d\n", a);

    printf("\nBefore callByReference: b = %d\n", b);

    callByReference(&b);

    printf("After callByReference: b = %d\n", b);

    return 0;

}

```

SAMPLE INPUT AND OUTPUT

Output:

Before callByValue: a = 5

Inside callByValue function: x = 15

After callByValue: a = 5

Before callByReference: $b = 5$

Inside callByReference function: $y = 15$

After callByReference: $b = 15$

RESULT

Thus, the C program to implement user-defined functions with different parameter passing techniques (Call by Value and Call by Reference) was successfully executed and verified.

Ex. No. : 7	Dynamic Memory Management using <code>malloc()</code>, <code>calloc()</code>, and <code>free()</code> in C

AIM

To write a C program that demonstrates dynamic memory allocation using `malloc()`, `calloc()`, and `free()` functions and performs operations on the allocated memory.

ALGORITHM

Algorithm (Using `malloc()`)

1. Start the program.
2. Declare a pointer variable and an integer variable `n` to store size.
3. Read the number of elements from the user.
4. Allocate memory using `malloc(n * sizeof(int))`.
5. Check if memory is successfully allocated.
6. Read elements from the user and store them in the allocated memory.
7. Display the elements.
8. Release the allocated memory using `free()`.
9. Stop the program.

Algorithm (Using `calloc()`)

1. Start the program.
2. Declare a pointer and an integer variable `n`.
3. Read the number of elements from the user.
4. Allocate contiguous memory using `calloc(n, sizeof(int))`.
5. Check if memory allocation is successful.
6. Since `calloc` initializes memory to zero, print the initial values.
7. Read elements from the user.
8. Display the elements.
9. Release memory using `free()`.
10. Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

int main() {

    int n, i;

    int *ptr1, *ptr2;

    // Using malloc()

    printf("Enter number of elements for malloc: ");

    scanf("%d", &n);

    ptr1 = (int *)malloc(n * sizeof(int));

    if (ptr1 == NULL) {

        printf("Memory not allocated using malloc.\n");

        exit(0);

    }

    printf("Enter %d elements:\n", n);

    for (i = 0; i < n; i++) {

        scanf("%d", &ptr1[i]);

    }

    printf("Elements stored using malloc:\n");

    for (i = 0; i < n; i++) {

        printf("%d ", ptr1[i]);

    }

    printf("\n");

    free(ptr1); // releasing memory

    // Using calloc()

```

```

printf("\nEnter number of elements for calloc: ");

scanf("%d", &n);

ptr2 = (int *)calloc(n, sizeof(int));

if (ptr2 == NULL) {

    printf("Memory not allocated using calloc.\n");

    exit(0);

}

printf("Initial values stored using calloc (should be 0):\n");

for (i = 0; i < n; i++) {

    printf("%d ", ptr2[i]);

}

printf("\nEnter %d elements:\n", n);

for (i = 0; i < n; i++) {

    scanf("%d", &ptr2[i]);

}


printf("Elements stored using calloc:\n");

for (i = 0; i < n; i++) {

    printf("%d ", ptr2[i]);

}

printf("\n");

free(ptr2); // releasing memory

```

```
    return 0;

}
```

SAMPLE INPUT & OUTPUT

Input

Enter number of elements for malloc: 4

Enter 4 elements:

10 20 30 40

Output

Elements stored using malloc:

10 20 30 40

Elements stored using malloc:

10 20 30 40

Elements stored using malloc:

10 20 30 40

Input

Enter number of elements for calloc: 5

Output

Initial values stored using calloc:

0 0 0 0 0

Enter 5 elements:

1 2 3 4 5

Elements stored using calloc:

1 2 3 4 5

RESULT

Thus, the program to demonstrate dynamic memory allocation using **malloc()**, **calloc()**, and **free()** was successfully executed and tested.

Ex. No. : 8	Structure

AIM

To write and execute a C program using **structures** to store and display information of students.

ALGORITHM

1. Start the program.
2. Define a structure named Student with members:
 - rollno (int)
 - name (string)
 - marks (float)
3. Declare an array of structures to store details of multiple students.
4. Read the number of students n.
5. Use a loop to read each student's details (roll number, name, and marks).
6. Display all stored student details using another loop.
7. Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
struct Student {
```

```
    int rollno;
```

```
    char name[50];
```

```
    float marks;
```

```
};
```

```
int main() {
```

```
    struct Student s[50];
```

```
    int n, i;
```

```

printf("Enter number of students: ");

scanf("%d", &n);

// Reading details

for (i = 0; i < n; i++) {

    printf("\nEnter details for student %d\n", i + 1);

    printf("Roll Number: ");

    scanf("%d", &s[i].rollno);

    printf("Name: ");

    scanf("%s", s[i].name);

    printf("Marks: ");

    scanf("%f", &s[i].marks);

}

// Displaying details

printf("\n--- Student Details ---\n");

for (i = 0; i < n; i++) {

    printf("\nStudent %d\n", i + 1);

    printf("Roll Number: %d\n", s[i].rollno);

    printf("Name: %s\n", s[i].name);

    printf("Marks: %.2f\n", s[i].marks);

```

```
}
```

```
return 0;
```

```
}
```

SAMPLE INPUT

Enter number of students: 2

Enter details for student 1

Roll Number: 101

Name: Arun

Marks: 87.5

Enter details for student 2

Roll Number: 102

Name: Priya

Marks: 92.0

Sample Output

--- Student Details ---

Student 1

Roll Number: 101

Name: Arun

Marks: 87.50

Student 2

Roll Number: 102

Name: Priya

Marks: 92.00

RESULT

Thus, the structure-based program was successfully implemented, executed, and the output was verified.

Ex. No. : 9	File operations: Reading, Writing, and Modifying a File using C

AIM

To write a C program that demonstrates file handling operations such as **writing data to a file**, **reading data from a file**, and **modifying the contents of an existing file** using standard library functions like fopen(), fprintf(), fscanf(), fputs(), fgets(), and fclose().

ALGORITHM

1. Writing to a File

1. Start the program.
2. Declare the required file pointer FILE *fp.
3. Open a file in **write mode** using fopen("data.txt","w").
4. Check if the file opened successfully. If not, print an error message and exit.
5. Write content into the file using fprintf() or fputs().
6. Close the file using fclose().

2. Reading from a File

1. Open the same file in **read mode** using fopen("data.txt","r").
2. Check if the file opened successfully.
3. Read content from the file using fgets() or fscanf().
4. Display the content on the screen.
5. Close the file.

3. Modifying a File (Appending Data)

1. Open the file in **append mode** using fopen("data.txt","a").
2. Check if the file opened successfully.
3. Append new data to the existing file content using fprintf() or fputs().
4. Close the file.
5. Reopen in read mode and display the updated content.

4. Stop the program

PROGRAM (C Program)

```
#include <stdio.h>

int main() {

    FILE *fp;

    char text[100];

    // --- Writing to a File ---

    fp = fopen("data.txt", "w");

    if (fp == NULL) {

        printf("Error opening file!\n");

        return 1;

    }

    fprintf(fp, "Welcome to File Handling in C.\n");

    fprintf(fp, "This is the initial content of the file.\n");

    fclose(fp);

    printf("File written successfully.\n");

    // --- Reading from the File ---

    fp = fopen("data.txt", "r");

    if (fp == NULL) {

        printf("Error opening file for reading!\n");
```

```

        return 1;

    }

    printf("\n--- File Content Before Modification ---\n");

    while (fgets(text, sizeof(text), fp) != NULL) {

        printf("%s", text);

    }

    fclose(fp);

    // --- Modifying (Appending to the File) ---

    fp = fopen("data.txt", "a");

    if (fp == NULL) {

        printf("Error opening file for appending!\n");

        return 1;

    }

    fprintf(fp, "This line is appended to the file.\n");

    fclose(fp);

    printf("\nNew content appended.\n");

    // --- Reading Updated File Content ---

    fp = fopen("data.txt", "r");

    printf("\n--- File Content After Modification ---\n");

```

```
while (fgets(text, sizeof(text), fp) != NULL) {  
  
    printf("%s", text);  
  
}  
  
fclose(fp);  
  
return 0;  
  
}
```

SAMPLE OUTPUT

File written successfully.

--- File Content Before Modification ---

Welcome to File Handling in C.

This is the initial content of the file.

New content appended.

--- File Content After Modification ---

Welcome to File Handling in C.

This is the initial content of the file.

This line is appended to the file.

RESULT

Thus, a C program was successfully developed to demonstrate **file operations** such as **writing, reading,** and **modifying (appending)** data using standard file handling functions.